



福建師範大學
FUJIAN NORMAL UNIVERSITY

《人工智能》

【第三章-K-邻近算法】

黄锐泓

本节要点

1. 算法概述
2. K值的选取
3. 距离度量
4. 快速索引



算法概述

K-邻近算法 (k-Nearest Neighbors)

是一种常用的分类或回归算法：

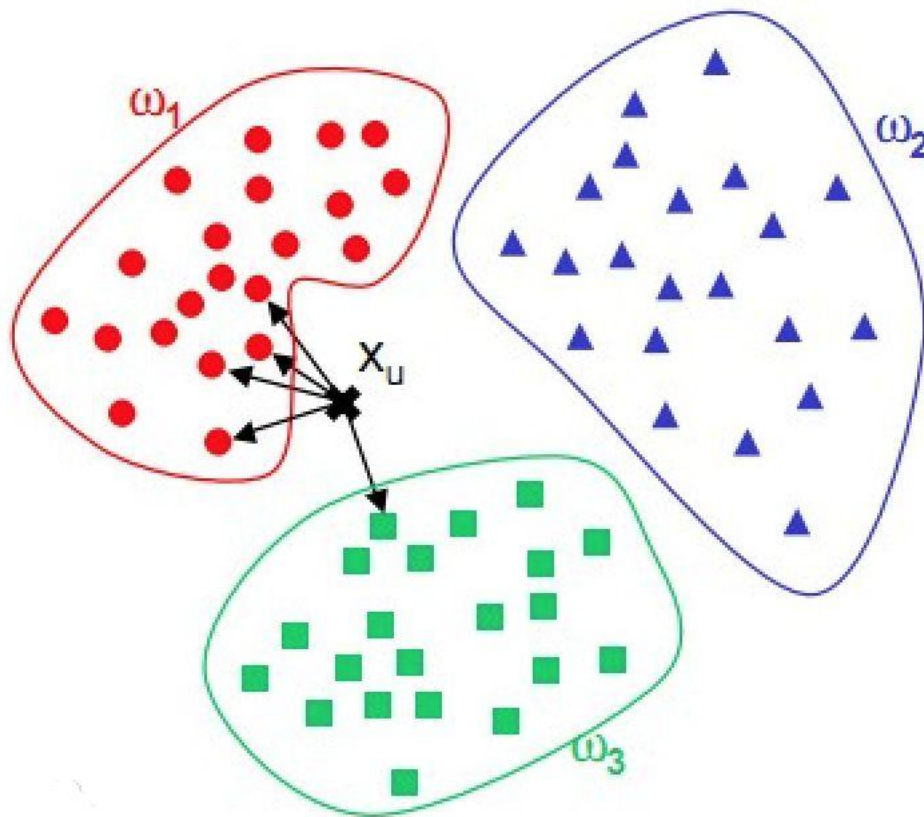
如果一个样本在特征空间中的 k 个最相似(即特征空间中最邻近)的样本中的大多数属于某一个类别，则该样本也属于这个类别。该方法在定类决策上只依据最邻近的一个或者几个样本的类别来决定待分样本所属的类别。

“物以类聚”



算法概述

依据离 X_u 距离最近的5个样本的类别情况，判断 X_u 的类别归属



算法概述

KNN 算法流程:

- 1、计算已知类别数据集中的点与待预测点之间的距离
- 2、按照距离递增次序排序
- 3、选取与待预测点距离最小的k个点
- 4、确定前k个点所在的类别的出现频率
- 5、返回前k个点出现频率最高的类别作为当前点的预测分类



K值的选取

1. 投票法(voting)

“少数服从多数原则”



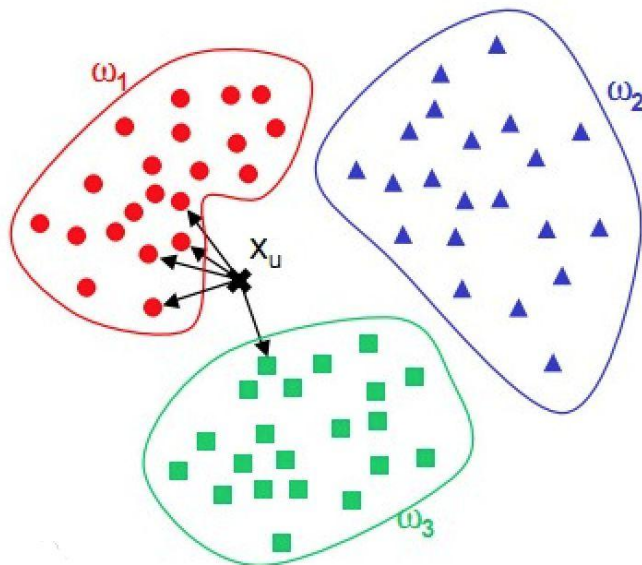
K值的选取

1. 投票法(voting)

“少数服从多数原则”

2. 两种极端情况

- a. $K=1$ 时，最邻近算法，即预测点的类别和最近的样本一致
- b. $K=n$ 时（ n 为所有样本数），会出现什么情况？



K值的选取

1. 投票法(voting)

“少数服从多数原则”

2. 两种极端情况

a. $K=1$ 时，最邻近算法，即预测点的类别和最近的样本一致

b. $K=n$ 时（ n 为所有样本数），会出现什么情况？

这时候只是简单预测为在训练实例中最多的类，模型过于简单



K值的选取

1. 投票法(voting)

“少数服从多数原则”

2. 两种极端情况

a. $K=1$ 时，最邻近算法，即预测点的类别和最近的样本一致

b. $K=n$ 时（ n 为所有样本数），会出现什么情况？

这时候只是简单预测为在训练实例中最多的类，模型过于简单

3. K的选择

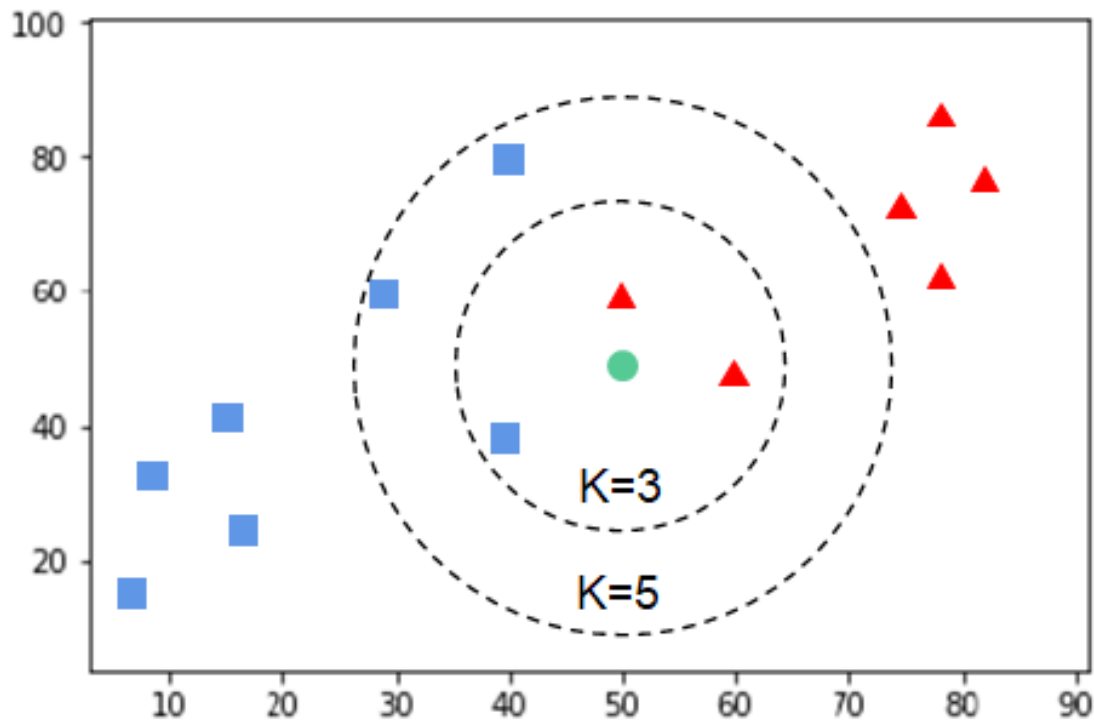
可以从 $K=1$ 开始，逐步增加，用检验数据来分析正确率，从而选择最优 K 。

K 增大，偏差逐渐增大而方差逐渐减少。

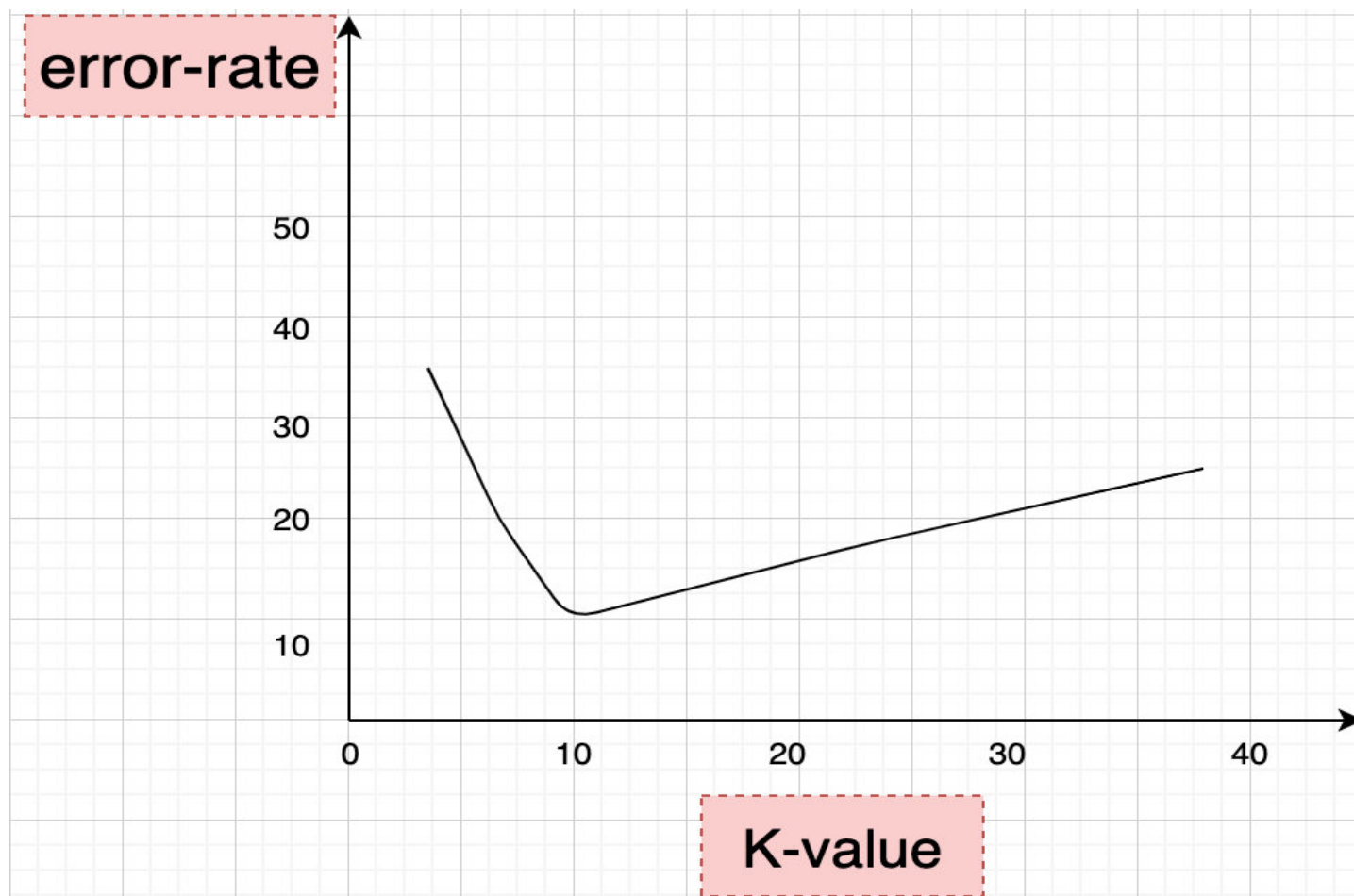


K值的选取

不同的K值，模型有时候会给出不同的结果



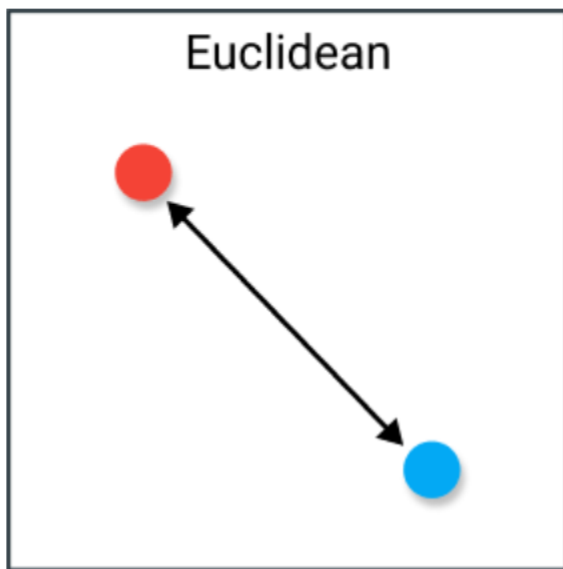
K值的选取



距离度量——KNN算法的核心

1. 距离度量方法

欧氏距离 (Euclidean Distance) $D(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$



距离度量——KNN算法的核心

1. 距离度量方法

欧氏距离 (Euclidean Distance) $D(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

在二维平面上假设有两个点 (x_1, x_2) 和 (y_1, y_2) , 则两点距离为:

$$D(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$



距离度量——KNN算法的核心

1. 距离度量方法

欧氏距离 (Euclidean Distance) $D(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

在二维平面上假设有两个点 (x_1, x_2) 和 (y_1, y_2) , 则两点距离为:

$$D(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

在三维空间上假设有两个点 (x_1, x_2, x_3) 和 (y_1, y_2, y_3) 则两点距离为:

$$D(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2}$$



距离度量——KNN算法的核心

1. 距离度量方法

欧氏距离 (Euclidean Distance) $D(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$



綜上易得，在高维空间中（n维）
，欧式距离可表示为：



距离度量——KNN算法的核心

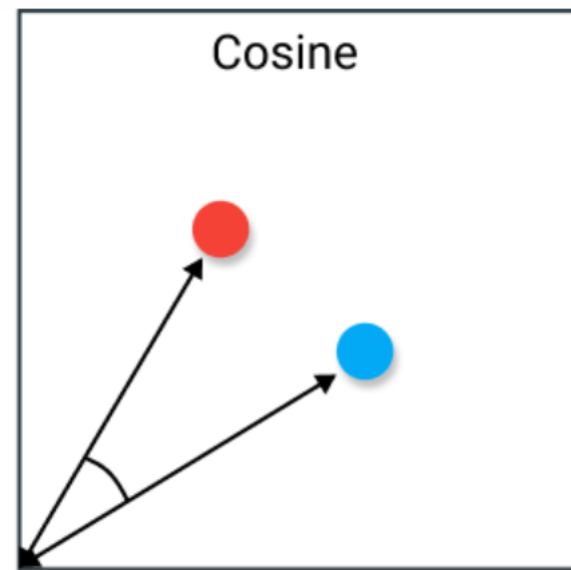
1. 距离度量方法

欧氏距离 (Euclidean Distance) $D(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

其他一些距离度量方法:

余弦相似度 (Cosine Similarity)
度量两个向量之间夹角的余弦

$$D(x, y) = \cos(\theta) = \frac{x \cdot y}{\|x\| \|y\|} = \frac{\sum_{i=1}^n (x_i y_i)}{\sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2}}$$



距离度量——KNN算法的核心

1. 距离度量方法

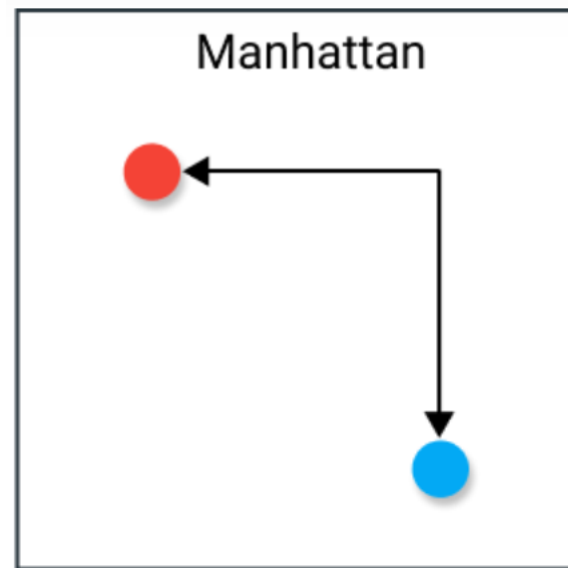
欧氏距离 (Euclidean Distance) $D(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

其他一些距离度量方法:

余弦相似度

曼哈顿距离 (Manhattan Distance)

$$D(x, y) = \sum_{i=1}^n |x_i - y_i|$$



距离度量——KNN算法的核心

1. 距离度量方法

欧氏距离 (Euclidean Distance) $D(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

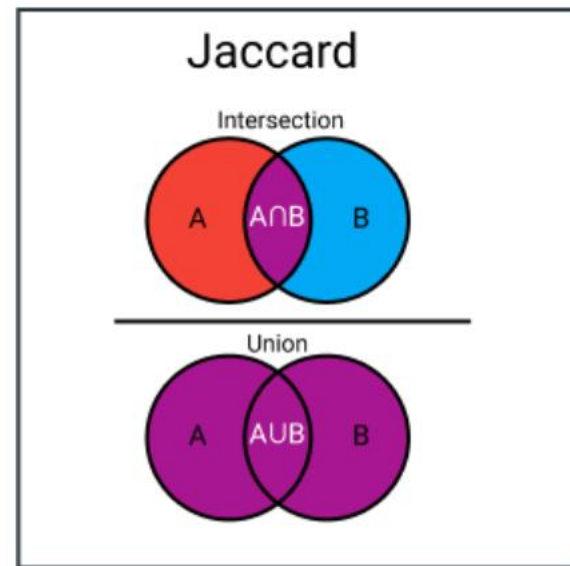
其他一些距离度量方法：

余弦相似度

曼哈顿距离

杰卡德距离与杰卡德指数 (Jaccard Index)

$$D(x, y) = 1 - \frac{|x \cap y|}{|x \cup y|}$$



距离度量——KNN算法的核心

两个点A和B的坐标分别为(1,2,3)、(4,5,6)如何计算他们的欧式距离、余弦相似度以及曼哈顿距离？



距离度量——KNN算法的核心

两个点A和B的坐标分别为(1,2,3)、(4,5,6)如何计算他们的欧式距离、余弦相似度以及曼哈顿距离？

欧氏距离：

$$D(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

$$D = \sqrt{(1-4)^2 + (2-5)^2 + (3-6)^2}$$

$$D = \sqrt{9 + 9 + 9}$$

$$D = \sqrt{27}$$

余弦相似度：

$$D(x, y) = \frac{\sum_{i=1}^n (x_i y_i)}{\sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2}}$$

$$D = \frac{(4+10+18)}{\sqrt{1+4+9} \sqrt{16+25+36}}$$

$$D = \frac{32}{\sqrt{14} \sqrt{77}}$$

曼哈顿距离：

$$D(x, y) = \sum_{i=1}^n |x_i - y_i|$$

$$D = |1-4| + |2-5| + |3-6|$$

$$D = 9$$



快速索引

当训练集合的规模很大时，如何快速的找到样本 x 的 k 个近邻成为计算机实现 k -近邻算法的关键。一个朴素的思想是：

- 1) 计算样本 x 与训练集中所有样本的距离。
- 2) 将这些点依据距离从小到大进行排序选择前 k 个。



快速索引

当训练集合的规模很大时，如何快速的找到样本 x 的 k 个近邻成为计算机实现 k -近邻算法的关键。一个朴素的思想是：

- 1) 计算样本 x 与训练集中所有样本的距离。
- 2) 将这些点依据距离从小到大进行排序选择前 k 个。

算法的时间复杂度是计算到训练集中所有样本的距离的时间加上排序的时间。



快速索引

一个更为可取的方法是为训练样本事先建立索引，以减少计算的规模。

kd树是一种典型的存储 k 维空间数据的数据结构（此处的 k 指 x 的维度大小，与 k -近邻算法中的 k 没有任何关系）。

建立好 kd 树后，给定新样本后就可以在树上进行检索，这样就能够大大降低检索 k 个近邻的时间，特别是当训练集的样本数远大于样本的维度时。



快速索引

KD树 (KD-Tree)

KD-Tree (K-Dimension Tree)，即一种将数据点在K维空间中进行划分的数据结构。中心思想：KD树能够将整个空间划分为特定的几个部分，只需要在特定空间中进行搜索操作，能够有效减少计算量。

前面的KNN的是排序后一次性搜索出K个最近的点，而这里介绍的KD-Tree则是搜索出最近的一个点后，得到K个最近点。

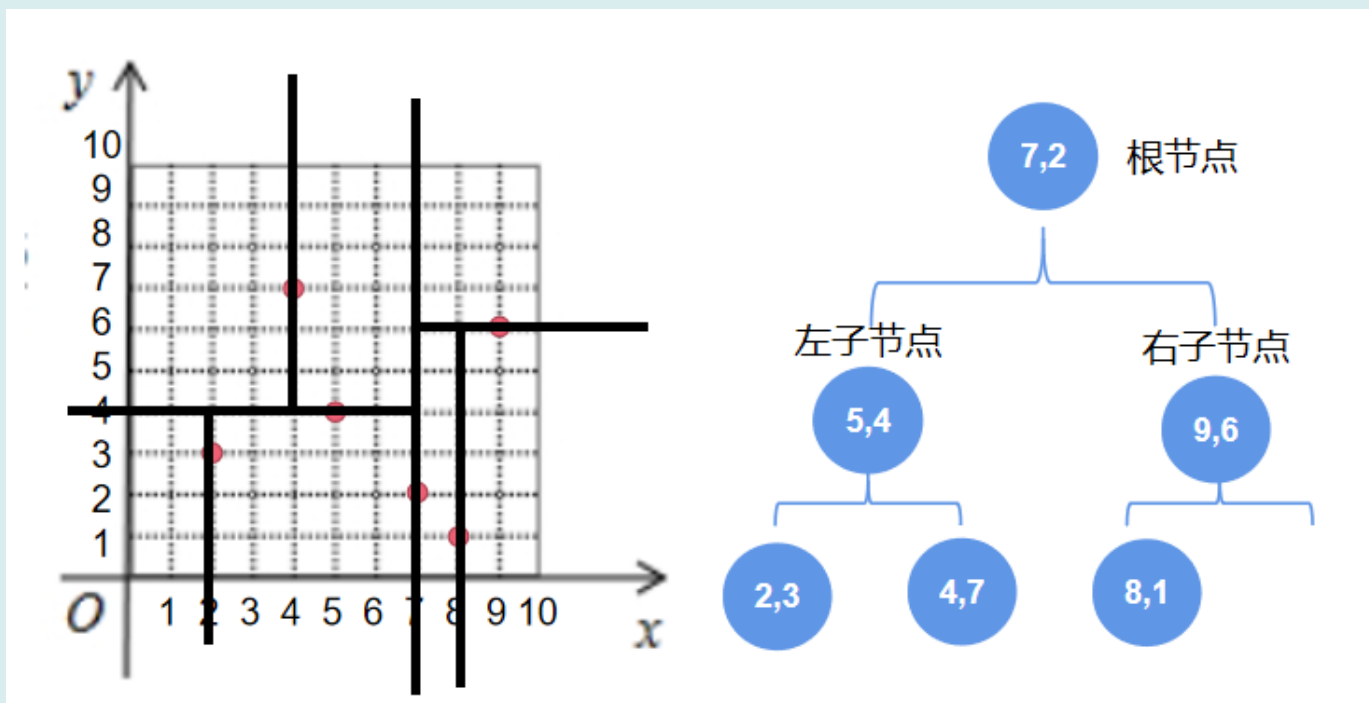


快速索引

我们使用下面这6个二维样本点进行KD-Tree的构建：

$(2,3)$ 、 $(5,4)$ 、 $(9,6)$ 、 $(4,7)$ 、 $(8,1)$ 、 $(7,2)$

最终可以得到下图的KD-Tree。左侧为二维空间的分割图，右侧为各个节点的关系图。



快速索引

前面提到KD-Tree (K-Dimension Tree) 是一种将数据点在K维空间中进行划分的数据结构。而上述的6个二维样本点，自然就只需要划分两个维度 (X、Y两个维度)

- 1) 默认选取x维度，以所有样本点在该维度上的数值进行升序排序，选取中位数对应的样本点为根节点，并将剩余样本点分别划分为根节点的左子节点和右子节点；
- 2) 再选取y维度，以左右子节点在该维度上的数值分别进行升序排序，分别选取中位数对应的样本点为根节点，并将剩余样本点分别划分为根节点的左子节点和右子节点；
- 3) 再选取x维度，如此反复进行，直到所有子根节点下都没有子节点了。

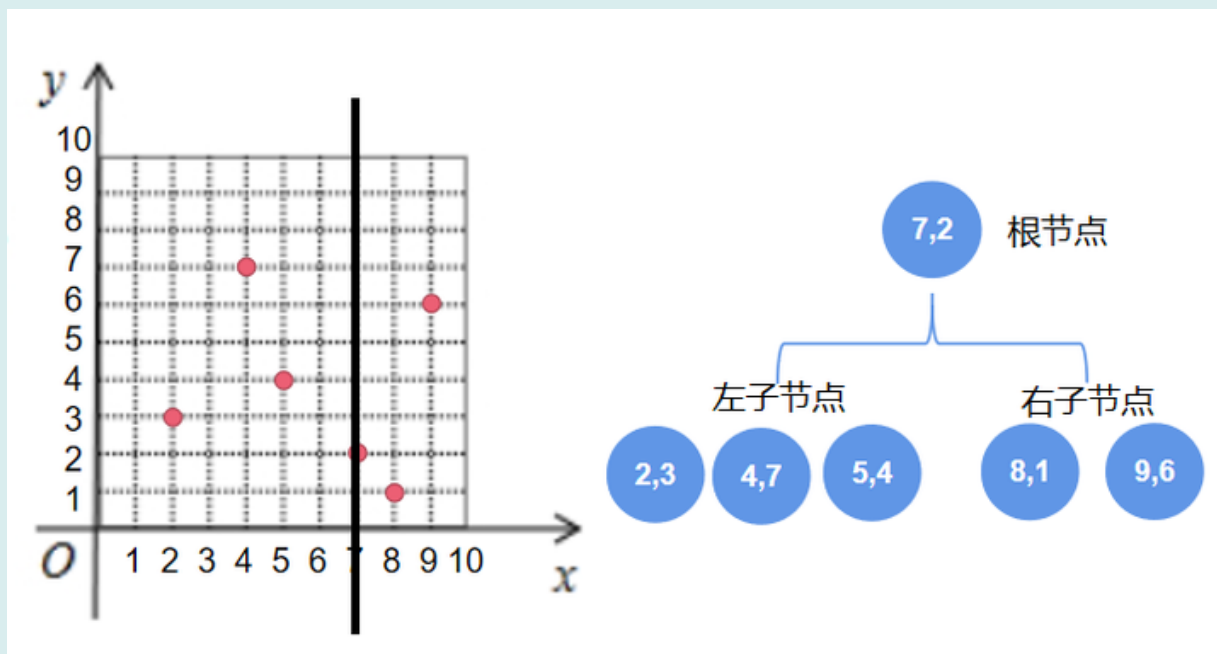


快速索引

对 $(2,3)$ 、 $(5,4)$ 、 $(9,6)$ 、 $(4,7)$ 、 $(8,1)$ 、 $(7,2)$ 在 x 维度上**升序**排序，可得：

$(2,3)$ 、 $(4,7)$ 、 $(5,4)$ 、 $(7,2)$ 、 $(8,1)$ 、 $(9,6)$

我们选取中间的一个样本点作为根节点， $(N/2+1) = 4$ ，即选取 $(7,2)$ 作为根节点。同时，我们将其左侧的点作为其左侧子节点，右侧点作为右侧子节点。

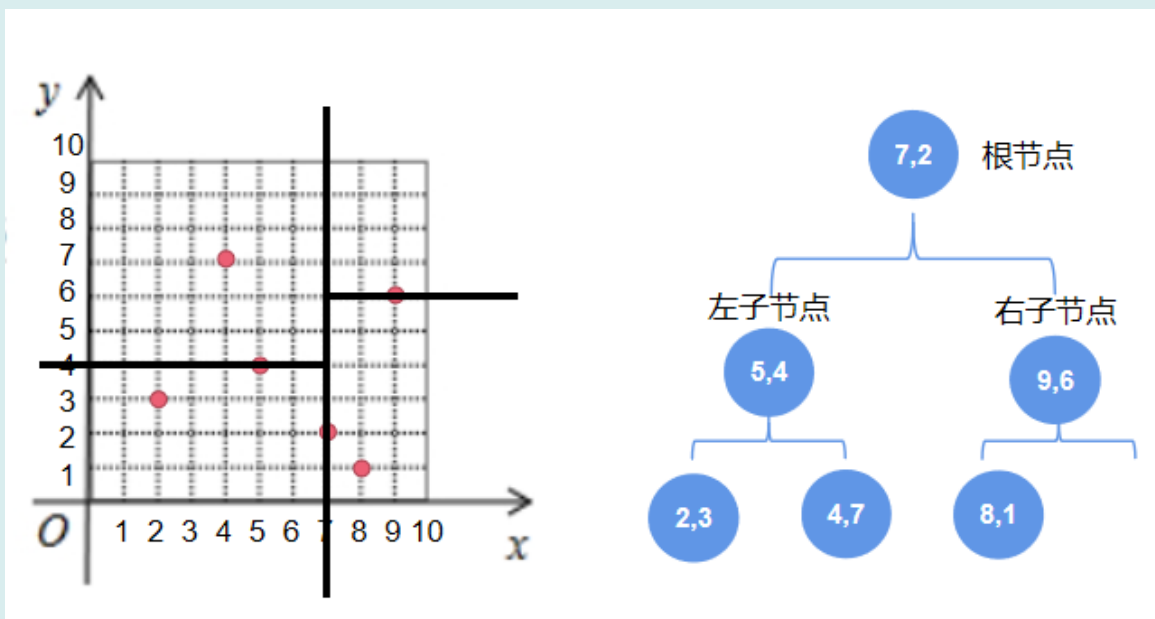


快速索引

经过上一步的划分后，我们进行子节点的划分。左右子节点需分别进行划分，这里以左侧子节点为例。按照y轴坐标进行排序，并选取中位数进行划分。

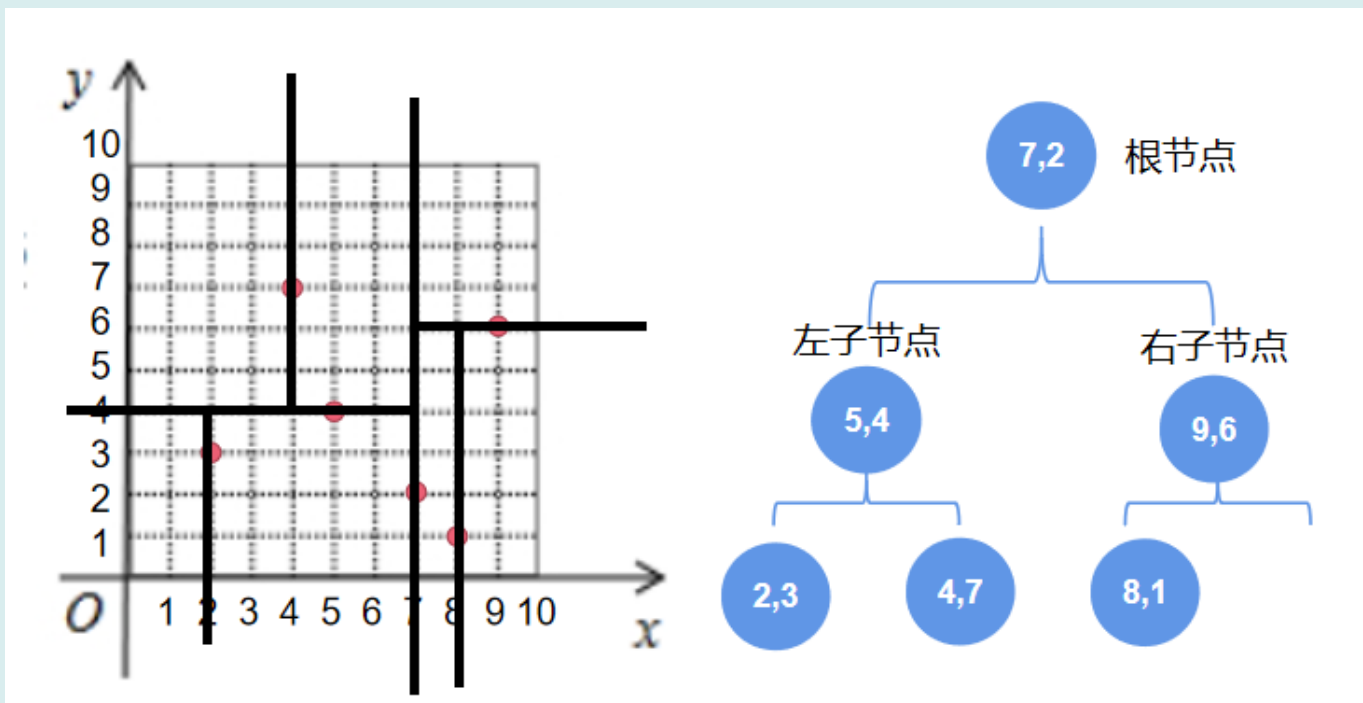
有三个左侧子节点， $N=3$ ，取中位数 $(N/2+1) = 2$ ，即选择 $(5,4)$ 作为左侧子节点的根节点。同时， $(2,3)$ 被划分为根节点 $(5,4)$ 的左侧子节点， $(4,7)$ 被划分为根节点的右侧子节点。

同理对 $(7,2)$ 的右侧子节点进行划分，可得下图



快速索引

经过前面两次的划分，我们已经将如上6个节点的关系划分完成。但左侧的图片中还需要进一步的划分子空间。即按照x轴进行划分，划分结果如下：



快速索引

kd 树搜索的过程如下：

1. 从根结点出发，递归地向下访问kd树。若预测样本 S 的坐标在当前维小于切分点，则向左访问，反之，向右访问，直到访问到叶节点，取该叶节点作为当前 S 的最近邻点。
2. 以 S 和当前最近邻点做圆。
3. 从当前叶节点回退父节点，并访问父节点的另一个分支，检查该节点或者子空间是否与圆相交
4. 若相交，则需要往下访问子节点，查找是否存在比当前最近邻点更近的节点
5. 若存在更近的样本，则更新最近邻点，并重复步骤2
6. 若不存在，则重复步骤3
7. 若不相交，则不需要访问这一分支，继续回退到上一层父节点，重复步骤3
8. 退回到根节点后结束搜索过程，并获得最近邻点。



快速索引

$(2, 3)$ 、 $(4, 7)$ 、 $(5, 4)$ 、 $(7, 2)$ 、 $(8, 1)$ 、 $(9, 6)$

输入点 $(2.5, 4.2)$

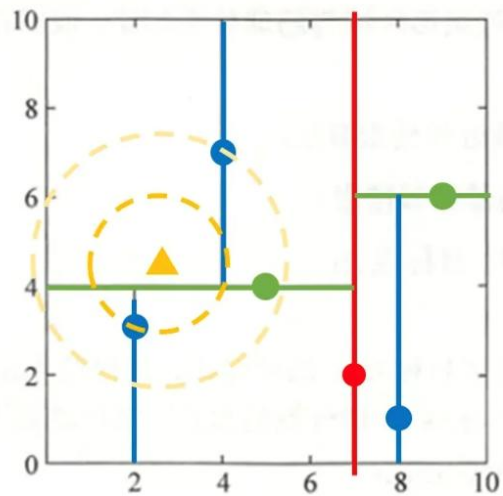
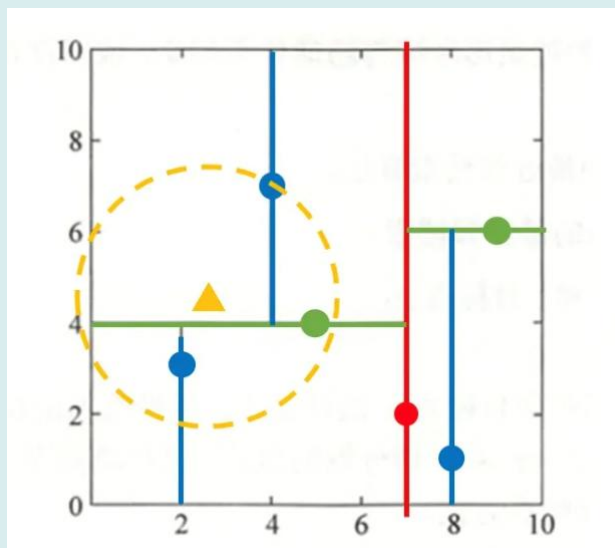


图 3.3 特征空间划分



小结

1. 算法概述
2. K值的选取
3. 距离度量（欧氏距离）
4. 快速索引



谢谢！

